

SunuPaie / AdiPAIE V1.7.1 — Guide de déploiement Windows Server 2022

Guide de déploiement SunuPaie / AdiPAIE V1.7 sur Windows Server 2022

Cible : serveur **grh** + **SQL Server Express 2025 Application** : SunuPaie (XAF Blazor Server, .NET 8) **Version** : V1.7.1 (mai 2026) — déploiement validé en prod ELTON **Auteur** : ELTON Oil Company / DSI **Changelog v3** : V1.7.1 unicité email Salarié (validation app + index unique filtré SQL) + script détection doublons. **Changelog v2** : ajout section **8bis** méthode officielle "base template", hotfix **Updater.cs** (**#if !RELEASE** retiré), pièges SSMS 22 vs 18.2 sur SQL 2025, fix login **IIS** **APPP00L\SunuPaiePool** post-restore, JSON escape dbconfig.

Table des matières

1. Pré-requis matériel & logiciel
 2. Installation IIS + WebSocket + .NET 8
 3. Installation SQL Server Express
 4. Configuration SQL Server pour SunuPaie
 5. Publication depuis Visual Studio
 6. Déploiement sur IIS
 7. Configuration connection string
 8. Premier lancement & initialisation 8bis. **Initialisation par restauration de base template (RECOMMANDÉ)**
 9. Sécurité, HTTPS & pare-feu
 10. Sauvegardes & maintenance
 11. Migration des données (go-live)
 12. Troubleshooting & erreurs fréquentes
 13. Mises à jour ultérieures
-

1. Pré-requis matériel & logiciel

Matériel minimum recommandé

Ressource	Minimum	Recommandé prod ELTON
CPU	2 vCPU	4 vCPU (Xeon ou équivalent)
RAM	4 GB	8 GB (SQL Express plafonné à 1 GB, le reste pour IIS/Blazor)
Disque	40 GB SSD	100 GB SSD (DB + backups + logs)
Réseau	1 Gbps	1 Gbps + IP fixe

OS & comptes

- **Windows Server 2022 Standard** (Build 20348+) à jour Windows Update
- Compte **Administrateur** local pour l'installation
- Compte **service** dédié pour le pool IIS (recommandé) :

```
New-LocalUser -Name "svc-sunupaie" -Description "Service account SunuPaie" -  
PasswordNeverExpires  
Add-LocalGroupMember -Group "IIS_IUSRS" -Member "svc-sunupaie"
```

Logiciels requis (à installer dans cet ordre)

1. **IIS 10** + features WebSocket / ASP.NET Core
 2. **ASP.NET Core 8.0 Hosting Bundle**
 3. **SQL Server 2022 Express** (gratuit, jusqu'à 10 GB par DB)
 4. **SQL Server Management Studio** (SSMS, optionnel mais utile)
-

2. Installation IIS + WebSocket + .NET 8

Script PowerShell complet (à lancer en admin)

```
# =====
# STEP 1 : IIS + WebSocket + ASP.NET Core hosting
# A lancer en ADMIN sur le serveur grh
# =====

$features = @(
    'Web-Server',                # IIS core
    'Web-WebServer',
    'Web-Common-Http',
    'Web-Default-Doc',
    'Web-Static-Content',
    'Web-Http-Errors',
    'Web-Http-Redirect',
    'Web-Health',
    'Web-Http-Logging',
    'Web-Log-Libraries',
    'Web-Request-Monitor',
    'Web-Performance',
    'Web-Stat-Compression',
    'Web-Dyn-Compression',
    'Web-Security',
    'Web-Filtering',
    'Web-Windows-Auth',          # Optionnel si SSO Windows
    'Web-App-Dev',
    'Web-Net-Ext45',
    'Web-ISAPI-Ext',             # CRITIQUE pour AspNetCoreModuleV2
    'Web-ISAPI-Filter',
    'Web-WebSockets',           # CRITIQUE pour Blazor SignalR
    'Web-Mgmt-Tools',
    'Web-Mgmt-Console',
    'NET-Framework-45-ASPNET',
    'NET-Framework-45-Features'
)

Install-WindowsFeature -Name $features -IncludeManagementTools -Restart:$false
Write-Host "▀ IIS + WebSocket installes" -ForegroundColor Green
```

ASP.NET Core 8.0 Hosting Bundle

△ Vérifier la dernière version sur <https://dotnet.microsoft.com/en-us/download/dotnet/8.0> (section ASP.NET Core Runtime → Hosting Bundle)

```
# Télécharger manuellement et installer en silent
$bundlePath = "C:\Temp\dotnet-hosting-bundle.exe"
# Après téléchargement manuel :
Start-Process -FilePath $bundlePath -ArgumentList '/quiet','/install','/norestart' -Wait

# Restart IIS pour activer le module
iisreset /restart
```

Vérifications post-installation

```
# 1. Service IIS démarre ?
Get-Service W3SVC

# 2. WebSocket installé ?
Get-WindowsFeature Web-WebSockets | Select Name, InstallState

# 3. AspNetCoreModuleV2 détecté ?
Import-Module WebAdministration
Get-WebGlobalModule | Where-Object { $_.Name -like 'AspNetCoreModuleV2*' }

# 4. .NET 8 runtime présent ?
dotnet --list-runtimes
# Doit montrer :
#   Microsoft.AspNetCore.App 8.0.x
#   Microsoft.NETCore.App 8.0.x
```

Test smoke IIS

Naviguer vers <http://grh> → la page par défaut IIS doit s'afficher.

3. Installation SQL Server Express

Téléchargement

URL officielle : <https://www.microsoft.com/fr-fr/sql-server/sql-server-downloads> → section **Express**
→ bouton **Télécharger maintenant**

Installation pas-à-pas (UI)

1. Lancer `SQL2022-SSEI-Expr.exe`
2. Type d'installation : **Personnalisé** (pour pouvoir activer Mixed Mode)
3. Choisir le dossier d'install (par défaut OK)
4. Lancer l'installation complète :
 - **Type d'installation** : Nouvelle installation autonome

- **Fonctionnalités** : Cocher uniquement **Database Engine Services**
 - **Configuration de l'instance** :
 - Instance **nommée** : **SQLEXPRESS** (par défaut, recommandé)
 - Sinon **par défaut** (instance **MSSQLSERVER**)
 - **Configuration du serveur** :
 - SQL Server Database Engine : démarrer en **Automatique**
 - Compte : **NT Service\MSSQL\$SQLEXPRESS** (par défaut OK)
 - **Configuration du moteur** :
 - Mode authentification : **Mixte** (Windows + SQL)
 - Mot de passe **sa** : **noter ce mot de passe** (à stocker dans le coffre-fort entreprise)
 - Ajouter l'admin Windows comme administrateur SQL
5. Terminer l'installation, **redémarrer le serveur** si demandé.

Installation SSMS (Management Studio)

URL : <https://aka.ms/ssmsfullsetup> Installer en local sur le serveur ou sur le poste de l'admin DBA.

4. Configuration SQL Server pour SunuPaie

Activer TCP/IP (si auth distante prévue)

Lancer **SQL Server Configuration Manager** :

1. **SQL Server Network Configuration** → **Protocols for SQLEXPRESS**
2. Activer **TCP/IP**
3. Onglet IP Addresses → tout en bas **IPAll** → **TCP Port = 1433**
4. Restart le service **SQL Server (SQLEXPRESS)**

Créer la base SunuPaie

Dans SSMS :

```
-- En tant qu'admin (sa ou Windows admin)
CREATE DATABASE SunuPaie
    COLLATE French_CI_AI; -- collation insensible casse + accents (FR)
GO

-- Verification
SELECT name, collation_name FROM sys.databases WHERE name = 'SunuPaie';

-- Creer un login dedie pour l'app
CREATE LOGIN sunupaie_app WITH PASSWORD = 'MotDePasseFort_2026_ELTON',
    CHECK_POLICY = ON, CHECK_EXPIRATION = OFF;

USE SunuPaie;
CREATE USER sunupaie_app FROM LOGIN sunupaie_app;
ALTER ROLE db_owner ADD MEMBER sunupaie_app; -- droits complets sur SunuPaie
GO
```

Bonne pratique : utiliser le login dédié `sunupaie_app` plutôt que `sa` dans la connection string de l'application.

Limites SQL Express à connaître

Limite	Valeur
Taille max DB	10 GB (suffisant pour ~10 ans de paie ELTON)
RAM utilisée par le moteur	1 GB max
CPU	1 socket / 4 cores max
SQL Server Agent	Non disponible (pas de jobs natifs)

Impact V1.7 : pas de SQL Agent → les crons (alertes fin mission, acquisition mensuelle congés) tournent via les `BackgroundService` .NET, déjà intégrés dans l'app SunuPaie. Aucune action requise.

5. Publication depuis Visual Studio

Préparation

```
# Sur le poste DEV
cd C:\Dev\AdiPAIE_V02
git pull origin dev # recuperer la derniere version
git log --oneline -3 # verifier les commits

# Build Release
dotnet build --no-incremental --configuration Release AdiPAIE_V02.sln
# Doit afficher : Build succeeded. 0 Error(s)
```

Création du profil de publication

1. Dans Visual Studio, **clic droit** sur `AdiPAIE_V02.Blazor.Server` → **Publier**
2. **Nouveau profil** :
 - Cible : **Dossier** (Folder publish)
 - Path : `C:\Dev\Publish_V1.7\`
 - Cliquer **Terminer**
3. Modifier les paramètres :
 - **Configuration** : `Release`
 - **Target Framework** : `net8.0`
 - **Deployment Mode** :
 - **Framework-Dependent** (recommandé — léger, ~50 MB, requiert .NET 8 sur grh)
 - OU **Self-Contained** si grh n'a pas .NET 8 (~150 MB, embarque le runtime)
 - **Target Runtime** : `win-x64`
 - Cocher **Produce single file** = NON (laisser décoché)
 - Cocher **Trim unused assemblies** = NON
4. Cliquer **Publier**

Vérification du package publié

Dans `C:\Dev\Publish_V1.7\` tu dois voir :

- `AdiPAIE_V02.Blazor.Server.exe` (point d'entrée)
- `AdiPAIE_V02.Blazor.Server.dll` + `AdiPAIE_V02.Module.dll` + DLLs DevExpress
- `dbconfig.json` ⚠ **À vérifier avant copie sur prod**
- `appsettings.json`
- Dossier `wwwroot/` (incluant `help/`, `css/`, `js/`)
- `web.config` (config IIS générée auto)

⚠ **Vérifier `dbconfig.json`** dans le package : la connection string ne doit PAS pointer sur ton SQL local de dev. La modifier avant copie ou directement après sur le serveur (voir étape 7).

6. Déploiement sur IIS

Création du pool d'application

Sur **grh**, en PowerShell admin :

```
Import-Module WebAdministration

$poolName = "SunuPaiePool"

# Si le pool existe deja, le supprimer
if (Test-Path "IIS:\AppPools\$poolName") {
    Stop-WebAppPool -Name $poolName
    Remove-WebAppPool -Name $poolName
}

# Créer le pool .NET 8 (No Managed Code = ASP.NET Core)
New-WebAppPool -Name $poolName
Set-ItemProperty -Path "IIS:\AppPools\$poolName" -Name managedRuntimeVersion -Value ""
Set-ItemProperty -Path "IIS:\AppPools\$poolName" -Name enable32BitAppOnWin64 -Value $false
Set-ItemProperty -Path "IIS:\AppPools\$poolName" -Name startMode -Value "AlwaysRunning"

# Identite : utiliser le compte de service dedie (recommande)
# Si tu n'en as pas, laisser ApplicationPoolIdentity (ligne suivante a sauter)
Set-ItemProperty -Path "IIS:\AppPools\$poolName" -Name processModel.identityType -Value 3
Set-ItemProperty -Path "IIS:\AppPools\$poolName" -Name processModel.userName -Value "grh\svc-sunupaie"
Set-ItemProperty -Path "IIS:\AppPools\$poolName" -Name processModel.password -Value "MotDePasseSvc"

# Recyclage : eviter le recycle automatique a 1740 min (29h)
Set-ItemProperty -Path "IIS:\AppPools\$poolName" -Name recycling.periodicRestart.time -Value "00:00:00"
# Recycle a 03h00 du matin (creneau hors prod)
Clear-ItemProperty -Path "IIS:\AppPools\$poolName" -Name recycling.periodicRestart.schedule
New-ItemProperty -Path "IIS:\AppPools\$poolName" -Name recycling.periodicRestart.schedule -Value @{value="03:00:00"}

Write-Host "▣ Pool $poolName cree" -ForegroundColor Green
```


Création du site IIS

```
$siteName = "SunuPaie"
$sitePath = "C:\inetpub\wwwroot\SunuPaie"

# Creer le dossier
New-Item -Path $sitePath -ItemType Directory -Force | Out-Null

# Donner les permissions au pool
icacls $sitePath /grant "IIS AppPool\${poolName}:(OI)(CI)RX" /T
# Si compte de service dedie :
# icacls $sitePath /grant "grh\svc-sunupaie:(OI)(CI)RX" /T

# Creer le site (HTTP par default, on configurera HTTPS apres)
if (Test-Path "IIS:\Sites\${siteName}") { Remove-Website -Name $siteName }
New-Website -Name $siteName `
    -PhysicalPath $sitePath `
    -ApplicationPool $poolName `
    -Port 80 `
    -HostHeader "" # Vide = repond a toutes les URLs sur port 80

Write-Host "▣ Site $siteName cree sur http://grh" -ForegroundColor Green
```

Copier les fichiers publiés

Depuis le poste DEV (avec accès UNC) :

```
# Copie via UNC (necessite acces admin sur grh)
Copy-Item -Path "C:\Dev\Publish_V1.7\*" `
    -Destination "\\grh\C$\inetpub\wwwroot\SunuPaie\" `
    -Recurse -Force

# Verification du timestamp
Get-Item "\\grh\C$\inetpub\wwwroot\SunuPaie\AdiPAIE_V02.Blazor.Server.dll" | Select
    Name, LastWriteTime
```

Alternative via RDP : copier-coller le dossier publié manuellement.

Démarrer le site

```
# Sur grh
Start-WebAppPool -Name "SunuPaiePool"
Start-Website -Name "SunuPaie"

# Verifier
Get-WebAppPoolState -Name "SunuPaiePool" # Started
Get-WebsiteState -Name "SunuPaie" # Started
```

7. Configuration connection string

Architecture — Où l'app lit-elle la connection string ?

SunuPaie utilise un **bootstrap intelligent** (`DbConfigHelper.cs`) avec **2 emplacements** :

Emplacement	Rôle	Survie aux MAJ ?
<code>C:\inetpub\wwwroot\SunuPaie\dbconfig.json</code>	Template copié à l'install (legacy bin path)	Écrasé à chaque déploiement
<code>C:\ProgramData\AdiPAIE_V02\dbconfig.json</code>	Production — chemin réel utilisé par l'app	Préservé entre les MAJ

Au 1er démarrage, si `ProgramData\AdiPAIE_V02\dbconfig.json` n'existe pas :

- L'app copie le template depuis `wwwroot\SunuPaie\dbconfig.json` vers `ProgramData`
- Cette copie est **idempotente** (ne se refait jamais)

À partir du 2ème démarrage, l'app lit toujours `ProgramData\AdiPAIE_V02\dbconfig.json` .

Permissions sur ProgramData

Le pool d'application IIS doit avoir **Read + Write** sur le dossier ProgramData :

```
# Créer le dossier (au cas ou)
New-Item -Path "C:\ProgramData\AdiPAIE_V02" -ItemType Directory -Force | Out-Null

# Donner les permissions au pool (compte ApplicationPoolIdentity OU compte de service)
icacls "C:\ProgramData\AdiPAIE_V02" /grant "IIS AppPool\SunuPaiePool:(OI)(CI)F" /T

# Si compte de service dédié :
# icacls "C:\ProgramData\AdiPAIE_V02" /grant "grh\svc-sunupaie:(OI)(CI)F" /T

# Verification
icacls "C:\ProgramData\AdiPAIE_V02"
```

Configurer la connection string

Option 1 — Pré-remplir directement dans ProgramData (recommandé prod)

Sur **grh**, créer manuellement le fichier `C:\ProgramData\AdiPAIE_V02\dbconfig.json` :

```
{
  "ConnectionStrings": {
    "ConnectionString": "Server=grh\\SQLEXPRESS;Database=SunuPaie;User
                        ID=sunupaie_app;Password=MotDePasseFort_2026_!
                        ELTON;TrustServerCertificate=true;Encrypt=false;Connection Timeout=30"
  }
}
```

⚠ Important sécurité :

- Au 1er démarrage de l'app, **DPAPI LocalMachine chiffre automatiquement le mot de passe** dans le fichier
- Après chiffrement, le fichier ressemblera à :

```
{ "ConnectionStrings": { "ConnectionString":
  "Server=...;Password=DPAPI:AQAAANCMnd8...=;..." } }
```

- Le mot de passe chiffré est lié à la **machine grh** : impossible de le déchiffrer sur un autre serveur (sécurité by design)
- Si tu changes de serveur, il faut remettre le mot de passe en clair, l'app le re-chiffrera au démarrage

Option 2 — Laisser le template dans bin et l'app fait la copie

1. Pré-éditer `C:\Dev\Publish_V1.7\dbconfig.json` AVANT de copier les fichiers
2. Le template sera copié automatiquement vers ProgramData au 1er lancement

Variantes selon ton setup

Cas	Connection string
Auth Windows + instance par défaut	Server=grh;Database=SunuPaie;Integrated Security=true;TrustServerCertificate=true
Auth Windows + instance SQLEXPRESS	Server=grh\\SQLEXPRESS;Database=SunuPaie;Integrated Security=true;TrustServerCertificate=true
Auth SQL + instance par défaut	Server=grh;Database=SunuPaie;User ID=sunupaie_app;Password=xxx;TrustServerCertificate=true
Auth SQL + instance SQLEXPRESS	Server=grh\\SQLEXPRESS;Database=SunuPaie;User ID=sunupaie_app;Password=xxx;TrustServerCertificate=true
TCP explicite (port 1433)	Server=tcp:grh,1433;Database=SunuPaie;User ID=sunupaie_app;Password=xxx;TrustServerCertificate=true

⚠ **Auth Integrated Security** + IIS : le compte du pool d'application doit avoir un login SQL associé.
Ajouter via SSMS :

```
CREATE LOGIN [grh\svc-sunupaie] FROM WINDOWS;
USE SunuPaie;
CREATE USER [grh\svc-sunupaie] FROM LOGIN [grh\svc-sunupaie];
ALTER ROLE db_owner ADD MEMBER [grh\svc-sunupaie];
```

Modifier la connection string en production (changement de mot de passe, par ex.)

```
# 1. Ouvrir le fichier de production
notepad C:\ProgramData\AdiPAIE_V02\dbconfig.json

# 2. Remettre le mot de passe en CLAIR (supprimer le prefixe DPAPI:)
# 3. Sauvegarder

# 4. Recycler le pool – l'app re-chiffre le mot de passe au prochain run
Restart-WebAppPool -Name "SunuPaiePool"
```

Verification post-config

```
# 1. Le fichier existe-t-il dans ProgramData ?
Test-Path "C:\ProgramData\AdiPAIE_V02\dbconfig.json"

# 2. Permissions correctes ?
icacls "C:\ProgramData\AdiPAIE_V02\dbconfig.json"

# 3. Le pool peut-il l'accéder ?
# → Tester via un navigateur sur http://grh, si l'app démarre sans erreur de connexion
DB c'est OK

# 4. Le mot de passe a-t-il bien été chiffré après le 1er run ?
Get-Content "C:\ProgramData\AdiPAIE_V02\dbconfig.json" | Select-String "DPAPI:"
# Doit retourner une ligne contenant DPAPI:xxxxx
```

Recycler le pool après modification

```
Restart-WebAppPool -Name "SunuPaiePool"
```

8. Premier lancement & initialisation

⚠ **MÉTHODE OFFICIELLE V1.7+ : initialisation par restauration d'une base template.** Voir section **8bis** ci-dessous. La méthode "tout-Updater" décrite ci-dessous reste documentée mais a montré ses limites en production (cf. retour terrain 2026-05-10).

Test d'accès

Dans le navigateur : <http://grh> → la page de login XAF doit s'afficher.

Ce qui se passe au 1er lancement (automatique XAF)

L'Updater XAF (`Updater.cs`) s'exécute automatiquement :

1. Crée toutes les tables (Salarie, Bulletin, Conjoint, **Enfant** V1.6, etc.)
2. Joue les migrations XPO (nouvelles colonnes V1.7 : `Salarie.Telephone` , `Conjoint.DateNaissance`)
3. Initialise les données de référence (CongeType, Convention, Categories, Echelons, etc.)
4. Crée le compte **Admin** par défaut (login : `Admin` , mot de passe : vide)
 - ⚠ **Bug historique corrigé en V1.7.0a** : avant ce hotfix, le bloc de création de l'Admin était entouré de `#if !RELEASE / #endif` dans `Updater.cs` → en prod (build Release) **aucun Admin ni rôle Administrators n'était créé**, conduisant à `Login failed for 'Admin'` . Le fix consiste à retirer ces directives. Si vous déployez une version antérieure à V1.7.0a, **utilisez impérativement la méthode 8bis** (restauration de base template).
5. Initialise les rôles GRH avec **les nouvelles permissions Bulletin/Salarie de V1.6.2**

Login Admin et premiers paramétrages

1. Login en **Admin** (changer le mot de passe immédiatement)
2. Aller dans **Paramètres globaux** :
 - Configurer SMTP (pour les notifications)
 - Configurer la société (Raison sociale, NINEA, RC, signataire)
3. **Init. rôles GRH** (bouton dans Paramètres) → message "X permission(s) ajoutée(s)"
4. Créer les **utilisateurs** :
 - **Administration** → **Utilisateurs** → **Nouveau**
 - Assigner **un seul rôle principal** par user (RH OU DAF OU AssistantRH...)
 - **Éviter de cumuler RH + RH_Manager** sur un même user (cf. issue V1.6.2 résolue)

Test smoke en RH

Login avec un user RH créé :

- Paie → Consultation bulletins → toutes les colonnes visibles (Matricule, Nom Complet, Statut badge coloré, Brut Fiscal, Brut Social, Net APayer)
- Gestion personnel → Annuaire famille → liste s'affiche
- Congés et absences → Provision annuelle (DAF) → calculs apparaissent
- Cliquer sur un salarié → fiche complète avec photo, KPI bandeau, badges colorés

8bis. Initialisation par restauration de base template (RECOMMANDÉ)

Méthode officielle V1.7+ pour le 1er déploiement client. Plus rapide, plus prévisible que l'auto-init XAF, et évite les pièges de seed silencieux.

Pourquoi cette méthode

L' **Updater** XAF doit faire en un seul démarrage : créer toutes les tables, jouer les migrations, semer les référentiels, créer Admin + rôles + permissions. En prod sur SQL Express 2025 + IIS, ce premier démarrage peut prendre plusieurs minutes et toute exception silencieuse laisse l'app dans un état inutilisable.

L'approche "**créer en dev, restaurer en prod**" déplace toute l'initialisation côté machine de développement (où on peut debugger F5) et ne fait en prod qu'une opération SQL Server triviale et idempotente.

Étape 1 — Créer la base "template" en dev

Dans SSMS local (machine de développement) :

```
USE master;
DROP DATABASE IF EXISTS SunuPaie_Template;
CREATE DATABASE SunuPaie_Template;
```

Étape 2 — Pointer l'app dev vers cette base

Modifier temporairement `appsettings.json` (ou la connection string `dbconfig.json` locale) pour pointer sur `SunuPaie_Template`.

Étape 3 — Lancer en DEBUG dans Visual Studio

Appuyer **F5** → l' **Updater** crée :

- Toutes les tables métier (Salarie, Bulletin, Conjoint, Enfant, etc.)
- Toutes les tables Permission (**PermissionPolicyUser** , **PermissionPolicyRole** , etc.)
- Le compte **Admin** + rôle **Administrators** + rôle **Employe**
- Les rôles GRH (RH, DAF, AssistantRH, RH_Manager) avec leurs permissions
- Les référentiels (CongeType, Convention, Categories, Echelons, Rubriques, etc.)
- Les données démo (10 salariés)

Vérification immédiate : login **Admin** / (vide) sur l'app dev → accès complet.

Étape 4 (optionnel) — Nettoyer les données démo

Si vous voulez une base "vierge de données opérationnelles" mais avec schéma + Admin + rôles + référentiels prêts pour un client neuf :

```
USE SunuPaie_Template;

-- Données opérationnelles à supprimer
DELETE FROM BulletinLigne;
DELETE FROM Bulletin;
DELETE FROM CongeDemande;
DELETE FROM SoldeConge;
DELETE FROM Pret;
DELETE FROM PretEcheance;
DELETE FROM ContratInterim;
DELETE FROM BulletinInterim;
DELETE FROM Enfant;
DELETE FROM Conjoint;
DELETE FROM Salarie;
DELETE FROM Site;
DELETE FROM UniteOrganisationnelle;

-- ⚠ NE PAS toucher à :
-- PermissionPolicyUser, PermissionPolicyRole, PermissionPolicy*
-- ModuleInfo, ModelDifference, ModelDifferenceAspect
-- Categories, Convention, CongeType, Echelon, Rubrique
-- ParametreGlobal, Departement, Fonction, etc.
```

Étape 5 — Backup .bak

```
BACKUP DATABASE SunuPaie_Template
TO DISK = 'C:\Temp\SunuPaie_Template_V1.7.bak'
WITH FORMAT, COMPRESSION, INIT,
NAME = 'SunuPaie_Template V1.7 - schema + Admin + roles + referentiels';
```

Le `.bak` fait typiquement 5 à 30 MB compressé.

Étape 6 — Transférer le .bak sur le serveur grh

Via partage réseau, RDP copier-coller, OneDrive, ou clé USB. Cible : `C:\Temp\SunuPaie_Template_V1.7.bak` sur le serveur.

Étape 7 — Restaurer sur grh

Sur le serveur, dans SSMS connecté à `grh\SQLEXPRESS` (ou via `sqlcmd`) :

```
USE master;

-- Si une SunuPaie existante doit être écrasée
ALTER DATABASE SunuPaie SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
DROP DATABASE SunuPaie;
GO

-- Vérifier le chemin physique (varie selon SQL Express 2022/2025)
SELECT physical_name FROM sys.master_files WHERE database_id = 1;
-- → ex. C:\Program Files\Microsoft SQL Server\MSSQL17.SQLEXPRESS\MSSQL\DATA\

-- Restore avec MOVE pour adapter les chemins logiques aux chemins physiques
RESTORE DATABASE SunuPaie
FROM DISK = 'C:\Temp\SunuPaie_Template_V1.7.bak'
WITH MOVE 'SunuPaie_Template'
    TO 'C:\Program Files\Microsoft SQL
        Server\MSSQL17.SQLEXPRESS\MSSQL\DATA\SunuPaie.mdf',
    MOVE 'SunuPaie_Template_log'
    TO 'C:\Program Files\Microsoft SQL
        Server\MSSQL17.SQLEXPRESS\MSSQL\DATA\SunuPaie.ldf',
REPLACE;

ALTER DATABASE SunuPaie SET MULTI_USER;
```

Étape 8 — Recréer le user IIS dans la base restaurée

Le RESTORE remet l'état exact de la base de dev → le mapping vers `IIS APPPOOL\SunuPaiePool` est devenu **orphan** (le SID ne correspond plus). À recréer :


```

USE SunuPaie;

-- Drop si existe (orphan)
IF EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'IIS
          APPPOOL\SunuPaiePool')
    DROP USER [IIS APPPOOL\SunuPaiePool];

-- Recréer attaché au login serveur
CREATE USER [IIS APPPOOL\SunuPaiePool] FOR LOGIN [IIS APPPOOL\SunuPaiePool];
ALTER ROLE db_owner ADD MEMBER [IIS APPPOOL\SunuPaiePool];

-- Vérification
SELECT dp.name AS user_name, r.name AS role_name
FROM sys.database_role_members rm
    INNER JOIN sys.database_principals dp ON rm.member_principal_id = dp.principal_id
    INNER JOIN sys.database_principals r ON rm.role_principal_id = r.principal_id
WHERE dp.name = 'IIS APPPOOL\SunuPaiePool';
-- Doit retourner : IIS APPPOOL\SunuPaiePool | db_owner

```

Étape 9 — Restart IIS et test login

```
Restart-WebAppPool -Name "SunuPaiePool"
```

Naviguer vers <http://grh/> → page de login XAF → **Admin** / (vide) → connecté.

Étape 10 — Sécurisation immédiate

1. **Changer le mot de passe Admin** dans Administration → Utilisateurs
2. Configurer SMTP, société, signataire dans Paramètres globaux
3. Créer les utilisateurs RH/DAF métier (un seul rôle par user)
4. Désactiver ou supprimer les éventuels users démo restants

Avantages de la méthode

- **Prévisible** : 100% du seed se fait en dev avec debugger
- **Reproductible** : le **.bak** est versionnable (Git LFS ou release artifacts)
- **Multi-clients** : un même **.bak** template sert pour N clients
- **Rapide en prod** : restore en quelques secondes vs init XAF de plusieurs minutes
- **Reversible** : rollback = re-restaurer le **.bak** template

Limites

- ⚠ Le **.bak** est lié à la **version SQL Server** : restaurer un **.bak** SQL 2022 sur SQL 2017 échoue. Toujours produire le **.bak** sur la même version SQL que la prod.
- ⚠ Pour les **upgrades de version applicative** (ex. V1.7 → V1.8), c'est l' **Updater** XAF qui prend le relais sur la base existante (cf. section 13).

9. Sécurité, HTTPS & pare-feu

Activer HTTPS (recommandé fortement)

Option A — Certificat auto-signé (test/intranet)

```
# Generer un cert auto-signé pour grh
$cert = New-SelfSignedCertificate `
    -DnsName "grh", "grh.elton.local" `
    -CertStoreLocation "cert:\LocalMachine\My" `
    -FriendlyName "SunuPaie HTTPS" `
    -NotAfter (Get-Date).AddYears(5) `
    -KeyAlgorithm RSA -KeyLength 2048

$thumbprint = $cert.Thumbprint
Write-Host "Cert thumbprint : $thumbprint"

# Bind sur le site IIS
New-WebBinding -Name "SunuPaie" -IPAddress "*" -Port 443 -Protocol "https"
$binding = Get-WebBinding -Name "SunuPaie" -Protocol "https"
$binding.AddSslCertificate($thumbprint, "My")

# Optionnel : forcer HTTPS (rediriger HTTP → HTTPS)
# Voir module URL Rewrite IIS
```

Option B — Certificat Let's Encrypt (production)

Utiliser **win-acme** : <https://www.win-acme.com/>

Option C — Certificat entreprise (fourni par DSI)

Importer le **.pfx** via **mmc → Certificats → Local Computer → Personal → Import**, puis bind comme option A.

Pare-feu

```
# Ouvrir port 80 (HTTP) et 443 (HTTPS)
New-NetFirewallRule -DisplayName "IIS HTTP" -Direction Inbound -Protocol TCP -LocalPort 80 -Action Allow
New-NetFirewallRule -DisplayName "IIS HTTPS" -Direction Inbound -Protocol TCP -LocalPort 443 -Action Allow

# Si SQL doit être accessible depuis d'autres serveurs
New-NetFirewallRule -DisplayName "SQL Server" -Direction Inbound -Protocol TCP -LocalPort 1433 -Action Allow
```

Hardening IIS (best practices)

```
# Cacher le header Server: Microsoft-IIS/...
Add-WebConfigurationProperty -Filter "system.webServer/security/requestFiltering" `
    -Name "removeServerHeader" -Value $true

# Reject TRACE / TRACK methods
Add-WebConfigurationProperty -Filter "system.webServer/security/requestFiltering/verbs" `
    -Name "." -Value @{verb='TRACE';allowed='false'}
```

10. Sauvegardes & maintenance

Sauvegarde DB quotidienne (Task Scheduler)

```
# Créer le dossier de backups
New-Item -Path "D:\Backups\SunuPaie" -ItemType Directory -Force

# Script de backup
@"
SqlCmd -S grh\SQLEXPRESS -E -Q "BACKUP DATABASE SunuPaie TO DISK = 'D:
    \Backups\SunuPaie\SunuPaie_$(Get-Date -Format yyyy-MM-dd_HH:mm).bak' WITH FORMAT,
    COMPRESSION, INIT"
"@ | Out-File -FilePath "C:\Scripts\Backup_SunuPaie.ps1" -Encoding utf8

# Tache planifiée quotidienne 02h00
$action = New-ScheduledTaskAction -Execute "PowerShell.exe" -Argument "-File C:
    \Scripts\Backup_SunuPaie.ps1"
$trigger = New-ScheduledTaskTrigger -Daily -At "02:00"
$principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -RunLevel Highest
Register-ScheduledTask -TaskName "SunuPaie Backup" -Action $action -Trigger $trigger -
    Principal $principal
```

Rétention 30 jours

```
# Supprimer les backups > 30 jours
Get-ChildItem -Path "D:\Backups\SunuPaie\*.bak" |
    Where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-30) } |
    Remove-Item -Force
```

Backup IIS config

```
# Mensuel
Backup-WebConfiguration -Name "Backup_$(Get-Date -Format yyyy-MM)"
```

Surveillance basique

```
# Verifier que le site est UP (a planifier toutes les 5 min)
$response = Invoke-WebRequest -Uri "http://grh" -UseBasicParsing -TimeoutSec 10 -
    ErrorAction SilentlyContinue
if ($response.StatusCode -ne 200) {
    Send-MailMessage -To "dsi@elton.sn" -Subject "SunuPaie DOWN" -Body "Code HTTP $
        ($response.StatusCode)"
}
```

11. Migration des données (go-live)

Au moment du go-live

⚠ **Pas de récupération automatique** depuis l'ancien système (JDE / paie historique). Voir la doc complète dans **Aide** → **Congés** → **Étape 6 et Migration prod.**

Imports disponibles dans SunuPaie

Import	Action UI	Service
Salariés	Paramètres → Importer salariés	<code>ImportSalarieService</code>
Comptes bancaires	Paramètres → Importer comptes bancaires	<code>ImportCompteBancaireService</code>
Conjoints	Paramètres → Importer conjoints	<code>ImportConjointService</code>
Soldes congés initiaux	V1.7.1 (à venir) — manuel SQL en attendant	(à coder)
Bulletins intérim	Paie → Wizard import bulletins intérim	<code>BulletinInterimImportService</code>

Script SQL pour init soldes congés (en attendant V1.7.1)

```
USE SunuPaie;

-- Pour 1 salarié exemple (à industrialiser via Excel + script PowerShell)
DECLARE @SalarieOid uniqueidentifier =
    (SELECT Oid FROM Salarie WHERE Matricule = '99001');
DECLARE @CongeTypeOid uniqueidentifier =
    (SELECT Oid FROM CongeType WHERE Code = 'CPAYE');
DECLARE @SoldeOid uniqueidentifier = NEWID();

INSERT INTO SoldeConge (Oid, Salarie, Annee, Type,
    JoursAcquis, JoursReportes, JoursPris, JoursEnAttente,
    Statut, DateCreation)
VALUES (@SoldeOid, @SalarieOid, YEAR(GETDATE()), @CongeTypeOid,
    0, 18.5, 0, 0,
    1, GETDATE());

INSERT INTO MouvementSolde (Oid, Solde, TypeMouvement, Jours,
    DateMouvement, Commentaire)
VALUES (NEWID(), @SoldeOid,
    5, -- MouvementSoldeType.Initialisation
    18.5,
    GETDATE(),
    'Migration JDE go-live AdiPAIE V1.7');
```

Liste des données à charger au go-live

Donnée	Source	Format	Volume estimé
Référentiels (Sites, Départements, Fonctions)	JDE / Excel	Saisie manuelle XAF	~50 lignes
Salariés	JDE export Excel	Import wizard	100+
Conjoints	JDE	Import wizard	~80
Enfants	JDE	Saisie manuelle (V1.6)	~250
Comptes bancaires	JDE	Import wizard	100+
Soldes congés	JDE	Script SQL ou V1.7.1	100+
Historique bulletins	NON migré	Reste dans JDE	—

12. Troubleshooting & erreurs fréquentes

Le site ne démarre pas — HTTP 500.30

Cause : ASP.NET Core Hosting Bundle pas installé OU mauvaise version

```
# Verification
dotnet --list-runtimes
# Doit montrer Microsoft.AspNetCore.App 8.0.x

# Si absent, réinstaller le Hosting Bundle
```

HTTP 500.19 — config error

Cause : `web.config` corrompu OU pool d'app mal configuré

```
# Verifier que le pool est en "No Managed Code"
Get-ItemProperty "IIS:\AppPools\SunuPaiePool" -Name managedRuntimeVersion
# Doit etre vide ""
```

Erreur connection SQL

A network-related or instance-specific error occurred while establishing a connection to SQL Server.

Causes possibles :

1. SQL Server pas démarré : `Start-Service MSSQL\$$SQLEXPRESS`
2. TCP/IP désactivé : SQL Server Configuration Manager
3. Mauvais nom d'instance dans dbconfig.json
4. Pare-feu : ouvrir port 1433
5. Login `sa` désactivé : utiliser `Set-LoginEnabled -LoginName 'sa' -Enabled $true`

WebSocket failed — Blazor n'arrive pas à se connecter

Symptôme : page blanche, errors console "WebSocket connection failed"

Cause : Module WebSocket pas activé sur IIS

```
# Verifier
Get-WindowsFeature Web-WebSockets
# Si "Removed" : Install-WindowsFeature -Name Web-WebSockets
```

Permissions RH — colonnes Bulletin invisibles

Cf. **issue V1.6.2 résolue** (cause = double rôle RH + RH_Manager). Solution :

```
USE SunuPaie;
DELETE ur
FROM PermissionPolicyUserUsers_PermissionPolicyRoleRoles ur
JOIN PermissionPolicyRole rhm ON rhm.Oid = ur.Roles AND rhm.Name = 'RH_Manager'
WHERE ur.Users IN (
    SELECT urh.Users
    FROM PermissionPolicyUserUsers_PermissionPolicyRoleRoles urh
    JOIN PermissionPolicyRole rh ON rh.Oid = urh.Roles AND rh.Name = 'RH'
);
```

Puis le user fait **logout + login**.

App lente au 1er accès après redémarrage

Normal : XAF compile les vues à la demande. Solutions :

1. Pool d'app en **AlwaysRunning** (déjà fait étape 6)
2. Préchauffage via **Initialization** IIS module (optionnel)

DB > 10 GB : SQL Express bloqué

Migration vers SQL Server Standard nécessaire. Coût licence Microsoft. En attendant : purger les vieux audits (**AuditDataItemPersistent** > 2 ans).

Login failed for 'Admin' sur base fraîche

Symptôme : Page de login XAF accessible, mais **Admin** / (*vide*) échoue avec **Login failed for 'Admin'. User name or password is incorrect.**

Cause racine (corrigée en V1.7.0a) : Dans **Updater.cs**, le bloc qui crée le rôle Administrators et l'utilisateur Admin était entouré de **#if !RELEASE / #endif** → en build prod (Release), ce code n'était pas compilé, donc les tables **PermissionPolicyUser** et **PermissionPolicyRole** restaient vides.

Vérification :

```
sqlcmd -S grh\SQLEXPRESS -d SunuPaie -E -C -Q "SELECT UserName FROM
PermissionPolicyUser"
```

- 0 ligne → bug confirmé. Solutions :

1. **Recommandée** : déployer la version V1.7.0a+ (le hotfix retire les directives) ET utiliser la **méthode 8bis** (restauration de base template).
2. **Alternative** : la méthode 8bis seule suffit même sur version antérieure puisque le **.bak** contient déjà l'Admin.

Erreur SSL avec ODBC Driver 18 (SQL Server 2025)

Symptôme :

```
Microsoft ODBC Driver 18 for SQL Server : SSL Provider:
The certificate chain was issued by an authority that is not trusted.
```

Cause : Le driver ODBC 18 force `Encrypt=true` ET `TrustServerCertificate=false` par défaut. Sur SQL Server Express 2025 avec son certificat auto-signé, la connexion est rejetée.

Solutions :

1. **Connection string app** (déjà appliqué dans `dbconfig.json`) :

```
Encrypt=true;TrustServerCertificate=true
```

2. **sqlcmd** : ajouter le flag `-C` pour Trust Server Certificate :

```
sqlcmd -S grh\SQLEXPRESS -d SunuPaie -E -C -Q "SELECT 1"
```

3. **SSMS** : préférer **SSMS 18.2** au lieu de SSMS 22 pour le quotidien. SSMS 18.2 utilise le driver legacy et se connecte sans avoir à cocher `Trust server certificate` à chaque connexion. Désinstallation SSMS 22 + install SSMS 18.2 résout immédiatement les soucis de TLS strict avec SQL 2025. À terme, SSMS 20+ supportera SQL 2025 nativement.

Login failed pour 'IIS APPPOOL\SunuPaiePool' (Error 4060)

Symptôme dans logs stdout :

```
fail: AlerteFinMissionInterimHostedService
Login failed for user 'IIS APPPOOL\SunuPaiePool'. (Error: 4060)
```

Cause : Le **login serveur** existe (créé en SQL via `CREATE LOGIN [IIS APPPOOL\SunuPaiePool] FROM WINDOWS`), mais le **user dans la base** SunuPaie n'existe pas (orphan ou jamais créé).

Fix :

```
USE SunuPaie;

IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'IIS
APPPOOL\SunuPaiePool')
    CREATE USER [IIS APPPOOL\SunuPaiePool] FOR LOGIN [IIS APPPOOL\SunuPaiePool];

ALTER ROLE db_owner ADD MEMBER [IIS APPPOOL\SunuPaiePool];

-- Vérification
SELECT dp.name AS user_name, r.name AS role_name
FROM sys.database_role_members rm
    INNER JOIN sys.database_principals dp ON rm.member_principal_id = dp.principal_id
    INNER JOIN sys.database_principals r ON rm.role_principal_id = r.principal_id
WHERE dp.name = 'IIS APPPOOL\SunuPaiePool';
```


⚠ Ce piège est **systématique après un RESTORE de base** (cf. méthode 8bis, étape 8) : le mapping login↔user est cassé par le restore et doit être recréé.

Erreur "Le mot clé n'est pas pris en charge : 'driver'"

Cause : Tentative d'utiliser un keyword **ODBC** dans une connection string **SqlClient** (l'app utilise `Microsoft.Data.SqlClient`, pas ODBC).

Mauvaise : `Driver={ODBC Driver 18 for SQL Server};Server=grh\SQLEXPRESS;...`

Bonne : `Server=grh\SQLEXPRESS;Database=SunuPaie;Integrated Security=true;Encrypt=true;TrustServerCertificate=true`

Doublons d'email Salarié — l'index unique ne se crée pas

Symptôme : à partir de V1.7.1, l' `Updater` tente de créer `UX_Salarie_Email` (index unique filtré). S'il y a des doublons d'email en base, la création échoue silencieusement (logs Debug uniquement). La protection app (`Salarie.OnSaving`) reste opérationnelle, mais l'intégrité SQL n'est pas garantie.

Détection préalable (à lancer AVANT de déployer V1.7.1) :

```
USE SunuPaie;

-- Doublons d'email parmi les salariés actifs (non null, non vides, non supprimés)
SELECT
    LOWER(LTRIM(RTRIM(Email))) AS EmailNormalise,
    COUNT(*) AS Nb,
    STRING_AGG(Matricule + ' - ' + FirstName + ' ' + LastName, ' | ') AS Salaries
FROM Salarie
WHERE Email IS NOT NULL
    AND Email <> ''
    AND GCRecord IS NULL
GROUP BY LOWER(LTRIM(RTRIM(Email)))
HAVING COUNT(*) > 1
ORDER BY Nb DESC;
```

Si doublons trouvés : corriger via UPDATE ciblés (vider l'email du mauvais salarié, ou attribuer un email distinct) :

```
-- Exemple : vider l'email du salarié 99025 qui partage celui du 99001
UPDATE Salarie SET Email = NULL WHERE Matricule = '99025';
```

Puis redémarrer le pool IIS → l' `Updater` créera l'index proprement.

JSON parse error 'S' invalid escape dans dbconfig.json

Cause : `Server=grh\SQLEXPRESS` dans un fichier JSON. Le `\S` est interprété comme une séquence d'échappement invalide.

Fix : doubler le backslash dans le JSON :

```
{
  "ConnectionStrings": {
    "ConnectionString": "Server=grh\\SQLEXPRESS;Database=SunuPaie;..."
  }
}
```

13. Mises à jour ultérieures

Procédure de mise à jour V1.7 → V1.8 (futures)

```
# 1. Sur le poste DEV
git pull origin dev
dotnet build --configuration Release AdiPAIE_V02.sln

# 2. Backup DB sur grh (avant tout déploiement)
SqlCmd -S grh\SQLEXPRESS -E -Q "BACKUP DATABASE SunuPaie TO DISK = 'D:\Backups\SunuPaie\Avant_V1.8.bak' WITH FORMAT, COMPRESSION"

# 3. Backup ProgramData (config production avec mdp chiffré DPAPI)
Copy-Item C:\ProgramData\AdiPAIE_V02\dbconfig.json `
  D:\Backups\dbconfig_V1.7_$(Get-Date -Format yyyyMMdd).json

# 4. Stop le pool sur grh
Stop-WebAppPool -Name "SunuPaiePool"

# 5. Backup du dossier IIS
Compress-Archive -Path C:\inetpub\wwwroot\SunuPaie\* -DestinationPath D:\Backups\SunuPaie_V1.7_$(Get-Date -Format yyyyMMdd).zip

# 6. Publier nouvelle version VS → Folder
# 7. Copier vers \\grh\C$\inetpub\wwwroot\SunuPaie\
#   △ Cela ECRASE le dbconfig.json template dans wwwroot, mais
#   PAS celui de C:\ProgramData\AdiPAIE_V02\ (qui reste la production)

# 8. Restart pool
Start-WebAppPool -Name "SunuPaiePool"

# 9. Smoke test https://grh
```

L'avantage du pattern ProgramData : ta connection string et ton mot de passe chiffré DPAPI **survivent automatiquement** à toute mise à jour. Pas besoin de re-saisir le mot de passe à chaque déploiement.

Rollback en cas de problème

```
# 1. Stop pool
Stop-WebAppPool -Name "SunuPaiePool"

# 2. Restaurer le ZIP
Remove-Item C:\inetpub\wwwroot\SunuPaie\* -Recurse -Force
Expand-Archive -Path D:\Backups\SunuPaie_V1.7_20260509.zip -DestinationPath C:\inetpub\wwwroot\SunuPaie\

# 3. Restaurer la DB si schema change
SqlCmd -S grh\SQLEXPRESS -E -Q "RESTORE DATABASE SunuPaie FROM DISK = 'D:\Backups\SunuPaie\Avant_V1.8.bak' WITH REPLACE"

# 4. Restart
Start-WebAppPool -Name "SunuPaiePool"
```

Support & contacts

- **Code source GitHub** : https://github.com/dienguis/AdiPAIE_V02
- **Documentation produit** : [wwwroot/help/](#) (accessible depuis l'app via icône Aide)
- **Architecture détaillée** : [docs/dashboards/MISSION_STATE.md](#)
- **Sources légales paie Sénégal** :
 - [AfricaPaieRH — Congés payés Sénégal](#)
 - Code du Travail Sénégal Loi 97-17 du 1er décembre 1997

Checklist finale go-live

- ☐ Pré-requis matériel validés (8 GB RAM, 100 GB SSD)
- ☐ Windows Server 2022 à jour Windows Update
- ☐ IIS + WebSocket installés (script PowerShell étape 2)
- ☐ ASP.NET Core 8 Hosting Bundle installé + vérifié ([dotnet --list-runtimes](#))
- ☐ SQL Server Express installé + TCP/I